Detection of Pneumonia from X-Ray images

Archana V S

Post Graduate certificate Program in Applied Data Science and AI, E&ICT IIT Roorkee

27-Jul-2025

Agenda

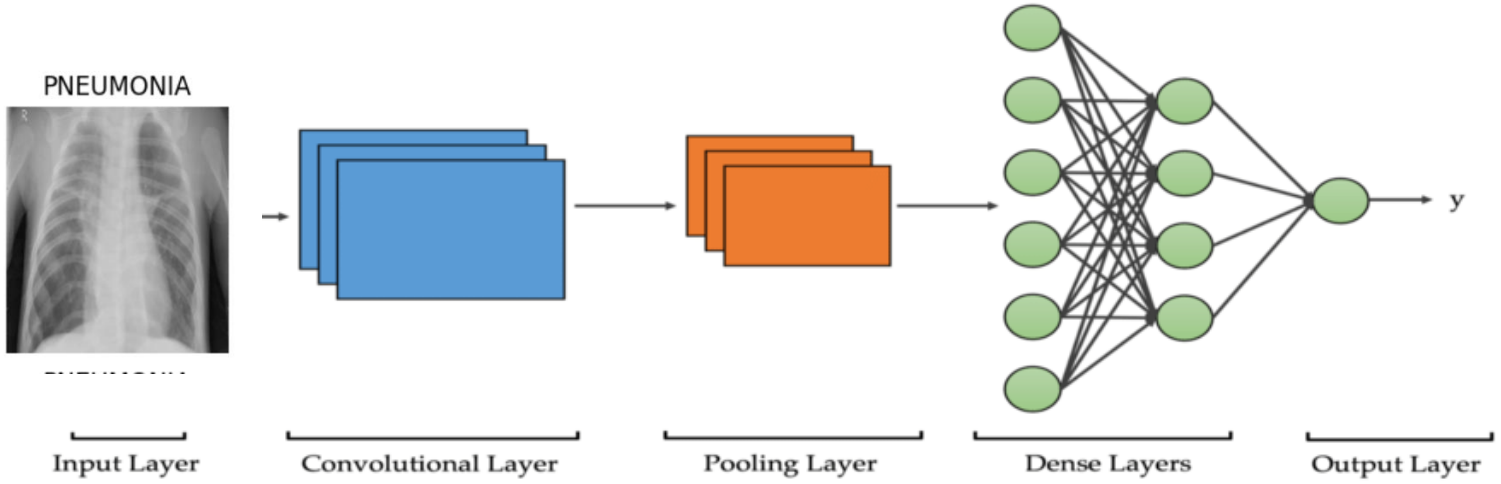
- Objective
- Approach to solving the problem
- Model summary
- Results
- Inference
- References
- Future developments

Objective

- Develop a deep learning model to classify X-Ray images for the detection of Pneumonia in a patient
- To detect the whether a given X-ray is predicted to be that of a Pneumonia infected individual.
- The data can be used to scan X-ray images to suggest whether further investigations are necessary for the patient and can be used by doctors and healthcare professionals or integrated with X-ray equipments as a value added feature to provide a preliminary assessment

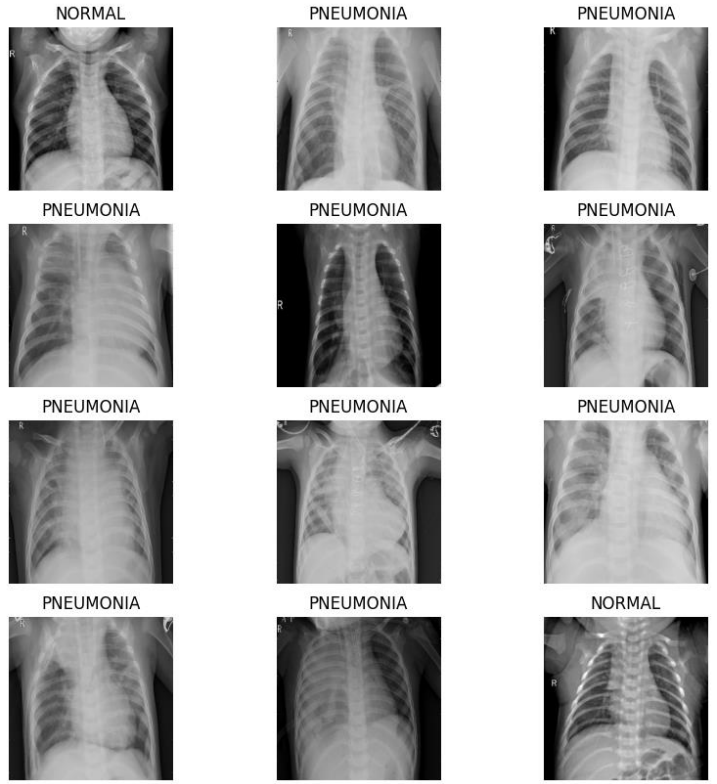
Approach to solving the problem

- This project uses Convolutional Neural Network (CNN) to create a model to classify X-Ray images based on whether Pneumonia is suspected or not.
- It uses the Keras Sequential API to solve the problem



Dataset

- The model is trained using 5216 input images out of which 1043 is used for validation
- The input images are roughly of the size 1917x1432 pixels. However the images are scaled down to the size 500x450 to reduce the number of parameters and training time
- Classification labels
  - NORMAL
  - PNEUMONIA



Model summary

Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling_1 (Rescaling)	(None, 500, 450, 3)	0
conv2d (Conv2D)	(None, 500, 450, 16)	448
max_pooling2d (MaxPooling2D)	(None, 250, 225, 16)	0
conv2d_1 (Conv2D)	(None, 250, 225, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 125, 112, 32)	0
conv2d_2 (Conv2D)	(None, 125, 112, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 62, 56, 64)	0
flatten (Flatten)	(None, 222208)	0
dense (Dense)	(None, 128)	28,442,752
dense_1 (Dense)	(None, 2)	258

Total params: 85,399,784 (325.77 MB)
Trainable params: 28,466,594 (108.59 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 56,933,190 (217.18 MB)

Results

- The training and validation accuracy and loss were promising, however the prediction accuracy on test data was only 76.41%. So the model was found to be overfitting.


```
# Accuracy of test predictions

1 print(accuracy)

0.7640625
```

Final Model – with augmentation and dropout

- With the original model, the training accuracy is increasing linearly over time close to 100% indicating overfitting.
- When there are a small number of training examples, the model sometimes learns from noises or unwanted details from training examples. To avoid this, generated additional training data from existing examples by augmenting them using random transformations that yield believable-looking images. This helps expose the model to more aspects of the data and generalize better.
- Also 20% dropout is used to reduce overfitting

Model Summary

Model: "sequential_2"

Layer (type)	Output Shape	Param #
sequential_1 (Sequential)	(None, 500, 450, 3)	0
rescaling_2 (Rescaling)	(None, 500, 450, 3)	0
conv2d_3 (Conv2D)	(None, 500, 450, 16)	448
max_pooling2d_3 (MaxPooling2D)	(None, 250, 225, 16)	0
conv2d_4 (Conv2D)	(None, 250, 225, 32)	4,640
max_pooling2d_4 (MaxPooling2D)	(None, 125, 112, 32)	0
conv2d_5 (Conv2D)	(None, 125, 112, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 62, 56, 64)	0
dropout (Dropout)	(None, 62, 56, 64)	0
flatten_1 (Flatten)	(None, 222208)	0
dense_2 (Dense)	(None, 128)	28,442,752
outputs (Dense)	(None, 2)	258

Total params: 85,399,784 (325.77 MB)
Trainable params: 28,466,594 (108.59 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 56,933,190 (217.18 MB)

Inference

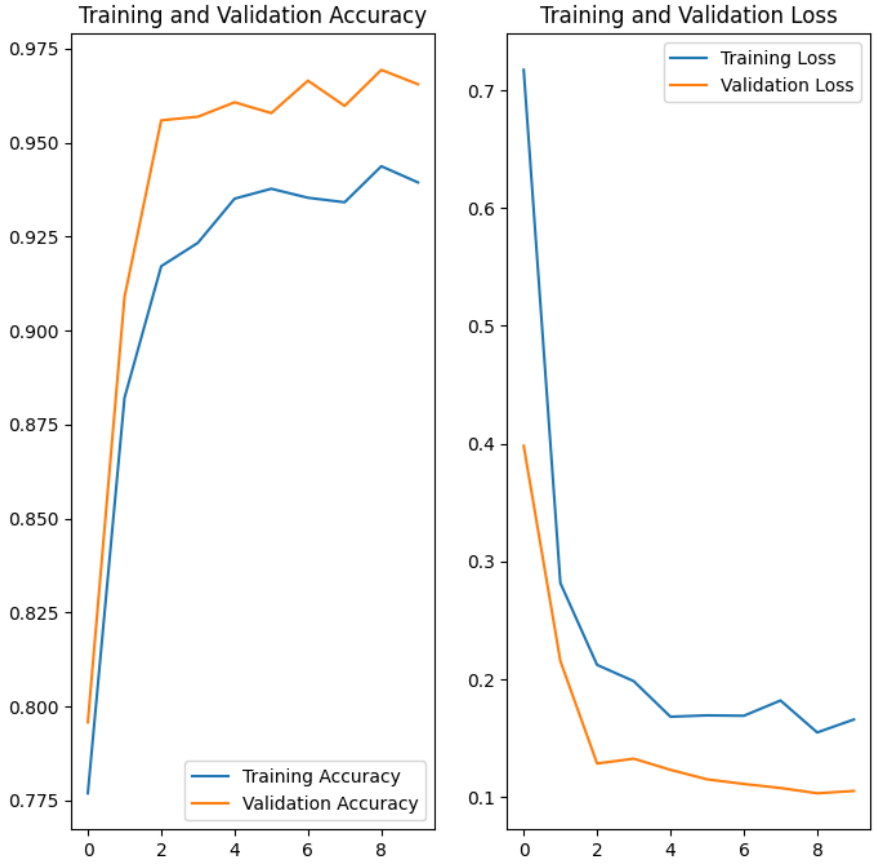
Results after adding data augmentation and dropouts

- The accuracy of cross validation is around 97% and loss is around 0.1
- The accuracy of predictions on new data increased to 78-82%

Accuracy of test predictions

```
print(accuracy)
```

```
0.821875
```



Future scope

- Training the images with different scale and dropout ratio
- Adding more data points will improve the reliability of the model.
- Straightening and trimming the images to the relevant content

References

- Code

https://github.com/archanavs/machinelrn/blob/master/xray_image_classification.ipynb

- Image classification tutorial from tensorflow

https://www.tensorflow.org/tutorials/images/classification#setup

- Chest X-ray dataset from Kaggle used as the dataset for this project

https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia

- Tensorflow API documentation

https://www.tensorflow.org/api_docs/python/tf